

Logic Gates, Logic Circuits & Boolean Algebra

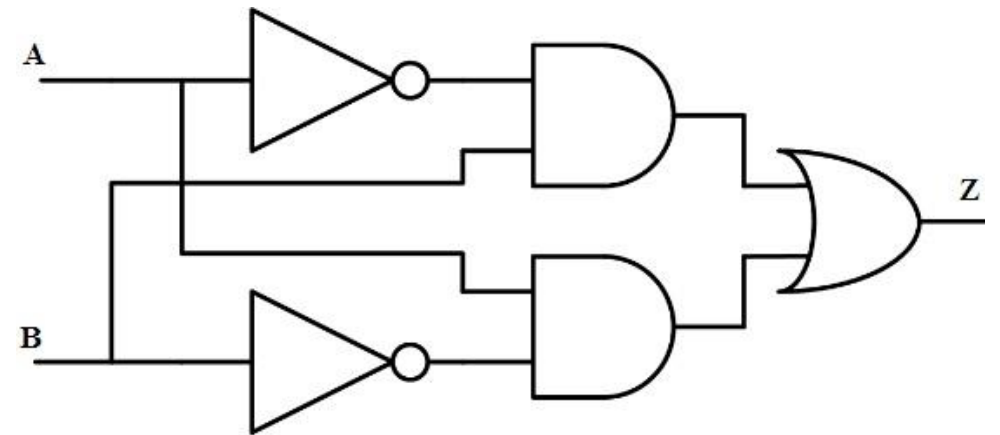
Lecture 9

Logic Gates

Logic gates are electronic circuits that operate on one or more input signals to produce standard output signal.

Some of the most basic and useful logic gates are -

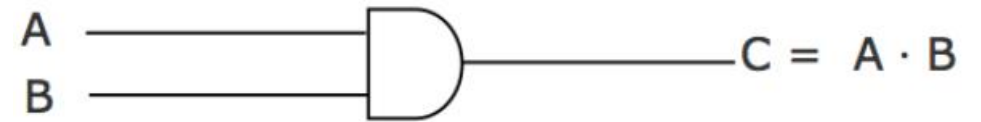
- AND,
- OR,
- NOT,
- NAND and
- NOR gate



AND Gate

Generates an output signal of 1 only if all input signals are also 1

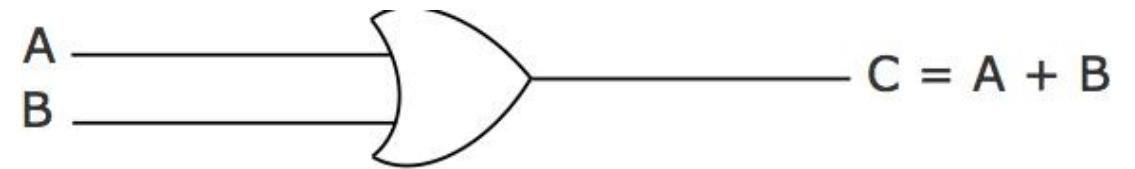
Inputs		Output
A	B	$C = A \cdot B$
0	0	0
0	1	0
1	0	0
1	1	1



OR Gate

Generates an output signal of 1 if at least one of the input signals is also 1

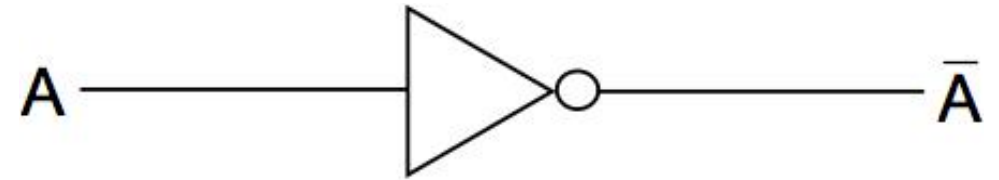
Inputs		Output
A	B	$C = A + B$
0	0	0
0	1	1
1	0	1
1	1	1



NOT Gate

Generates an output signal, which is the reverse of the input signal

Input	Output
A	$\bar{A}$
0	1
1	0

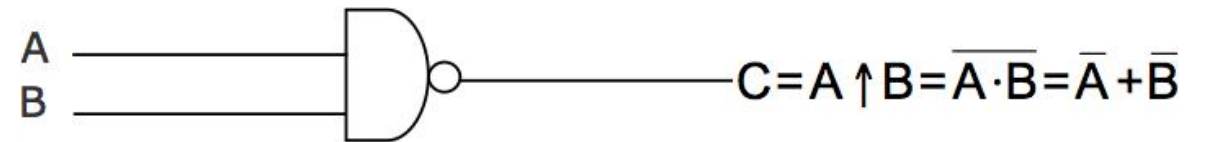


NAND Gate

Complemented AND gate. Generates an output signal of:

- ① 1 if any one of the inputs is a 0
- ② 0 when all the inputs are 1

Inputs		Output
A	B	$C = \bar{A} + \bar{B}$
0	0	1
0	1	1
1	0	1
1	1	0

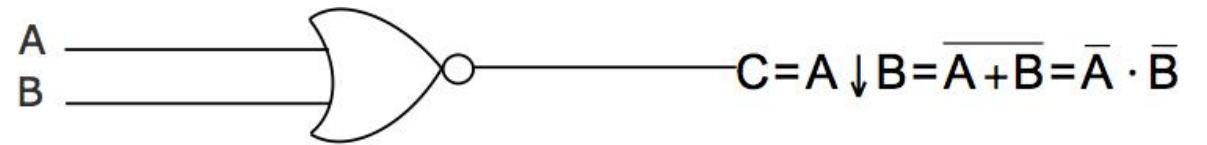


NOR Gate

Complemented OR gate. Generates an output signal of:

- ① 1 only when all inputs are 0
- ② 0 if any one of inputs is a 1

Inputs		Output
A	B	$C = \bar{A} \cdot \bar{B}$
0	0	1
0	1	0
1	0	0
1	1	0

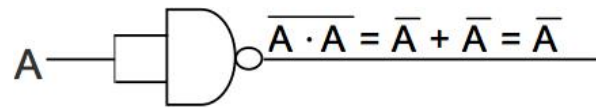


Universal NAND Gate

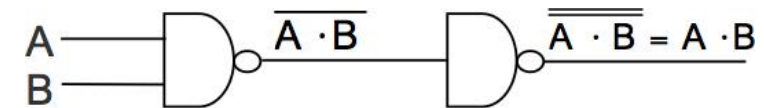
NAND gate is an universal gate, it is alone sufficient to implement any Boolean expression

To understand this, consider:

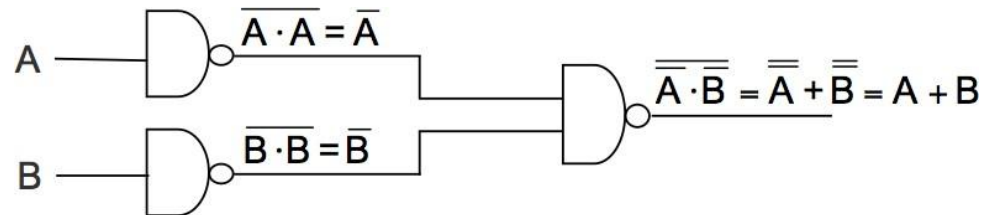
Basic logic gates (AND, OR, and NOT) are logically complete. Sufficient to show that AND, OR, and NOT gates can be implemented with NAND gates



(a) NOT gate implementation.



(b) AND gate implementation.



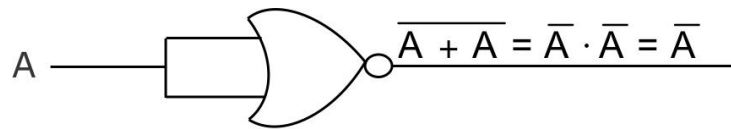
(c) OR gate implementation.

Universal NOR Gate

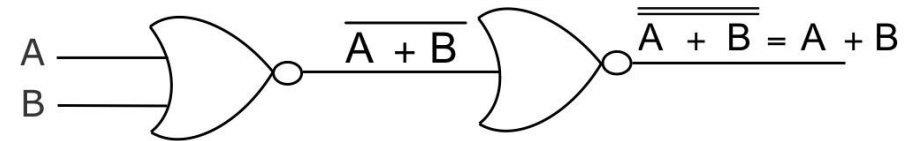
NOR gate is an universal gate, it is alone sufficient to implement any Boolean expression

To understand this, consider:

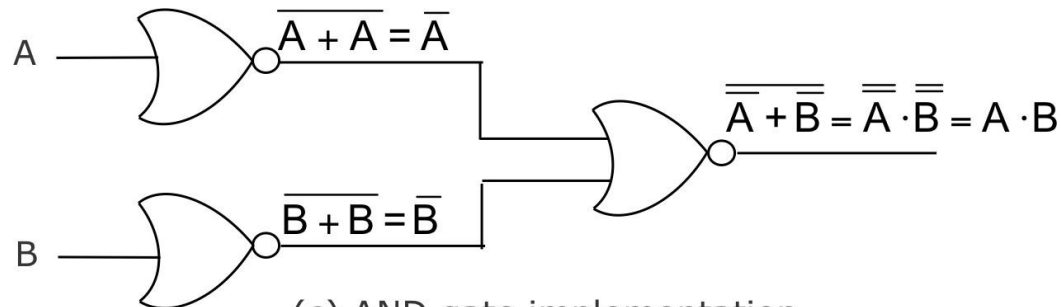
Basic logic gates (AND, OR, and NOT) are logically complete. Sufficient to show that AND, OR, and NOT gates can be implemented with NOR gates



(a) NOT gate implementation



(b) OR gate implementation.



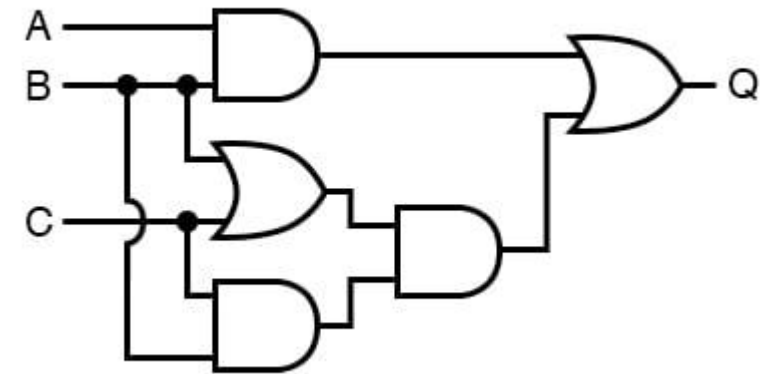
(c) AND gate implementation.

Logic Circuits

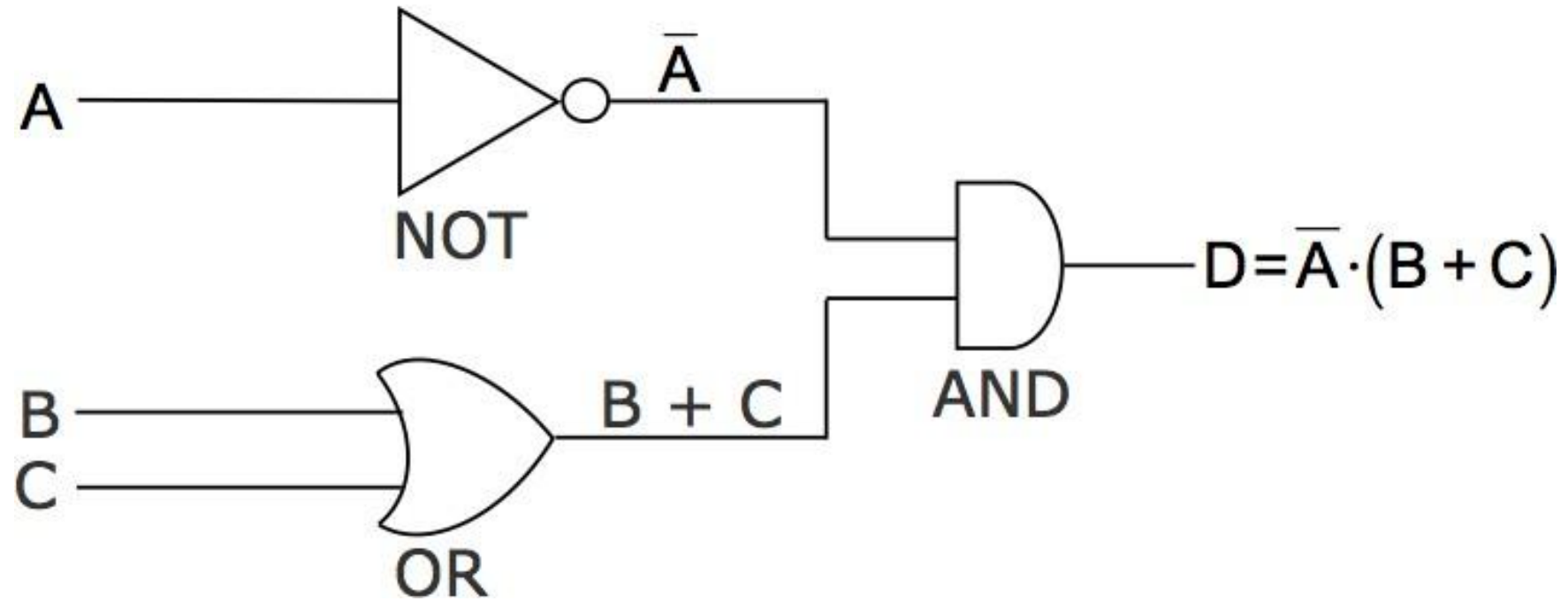
When logic gates are interconnected to form a gating / logic network, it is known as a combinational logic circuit

The Boolean algebra expression for a given logic circuit can be derived by systematically progressing from input to output on the gates

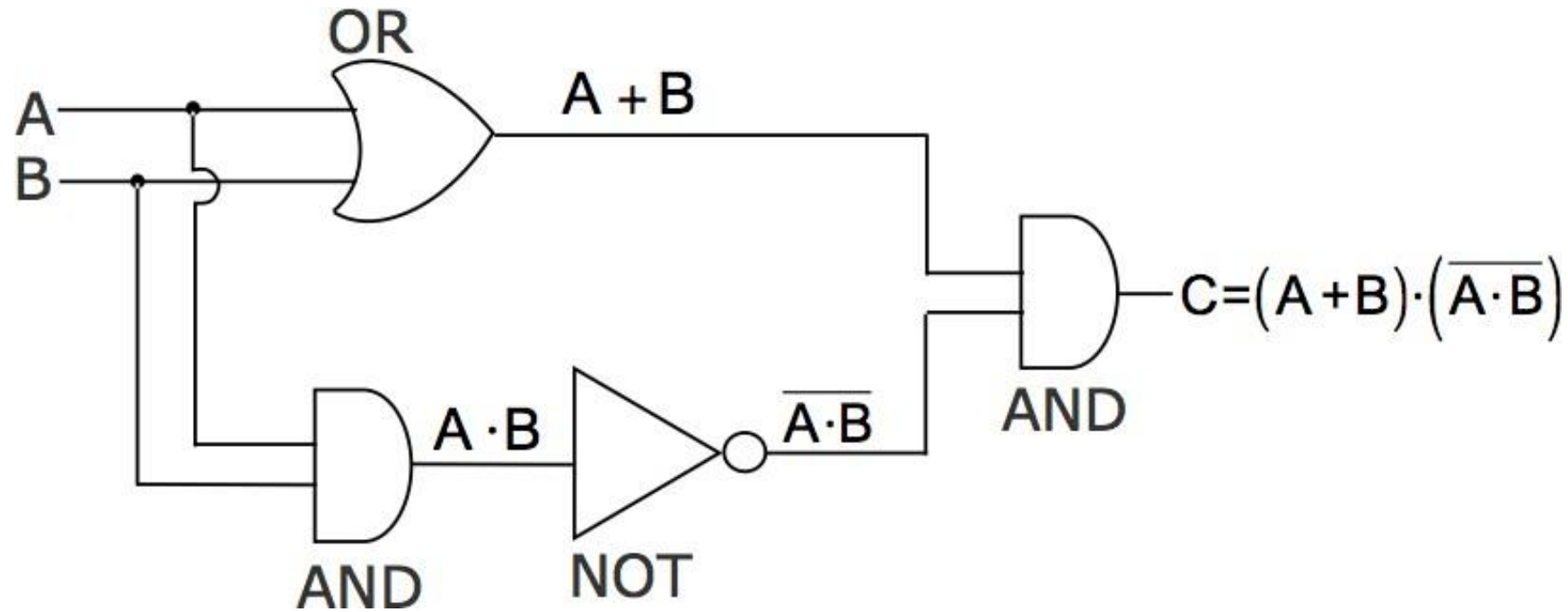
The three logic gates (AND, OR, and NOT) are logically complete because any Boolean expression can be realized as a logic circuit using only these three gates



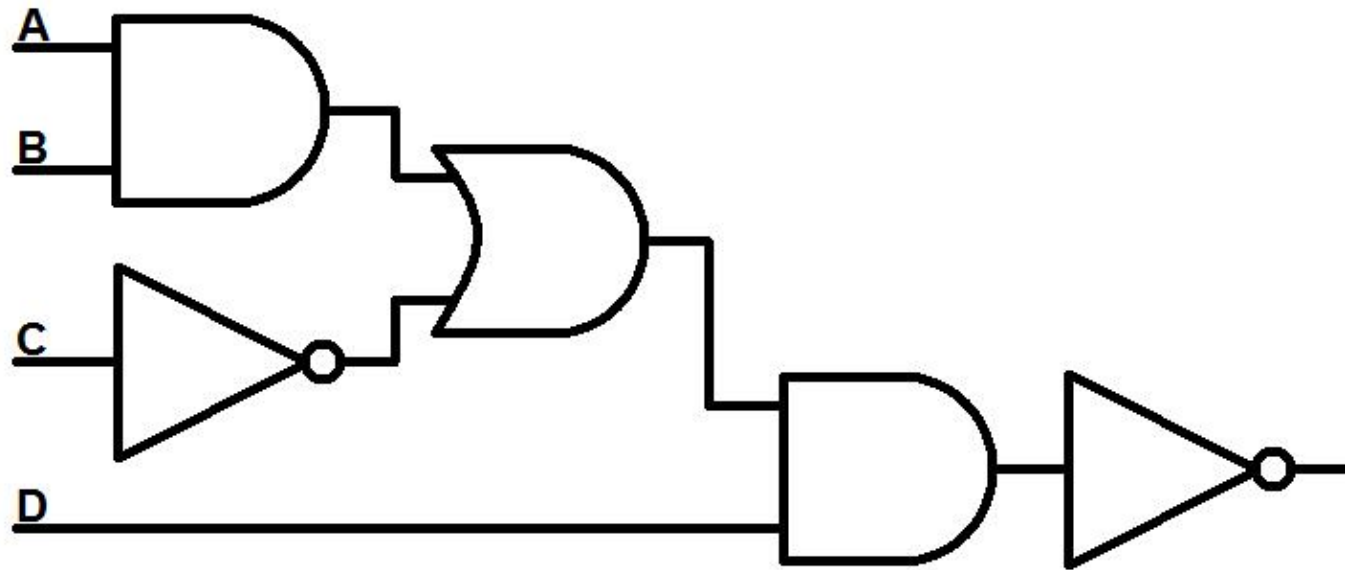
Finding Boolean Expression of a Logic Circuit (Example 1)



Finding Boolean Expression of a Logic Circuit (Example 2)



Finding Boolean Expression of a Logic Circuit (Example 3)

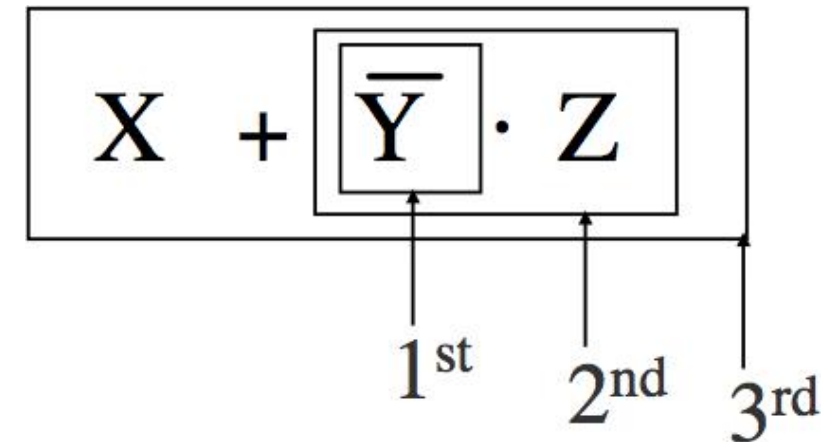


Boolean Algebra

- **Use of Binary Digit:** Boolean equations can have either of two possible values, 0 and 1
- **Logical Addition:** Symbol '+', also known as 'OR' operator, used for logical addition. Follows law of binary addition
- **Logical Multiplication:** Symbol '.', also known as 'AND' operator, used for logical multiplication. Follows law of binary multiplication
- **Complementation:** Symbol '-', also known as 'NOT' operator, used for complementation. Follows law of binary complement

Operator Precedence

- Each operator has a precedence level
- Higher the operator's precedence level, earlier it is evaluated
- Expression is scanned from **left to right**
- First, expressions enclosed within **parentheses** are **evaluated**
- Then, all complement (**NOT**) operations are performed
- Then, all '·' (**AND**) operations are performed
- Finally, all '+' (**OR**) operations are performed



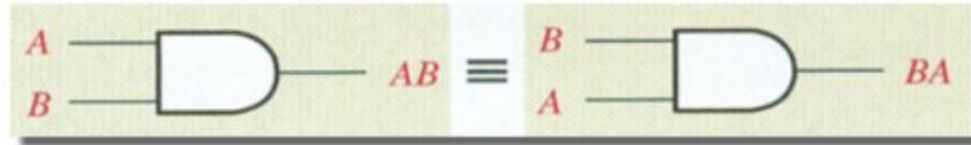
Laws of Boolean Algebra

◆ Commutative Laws

$$A + B = B + A$$

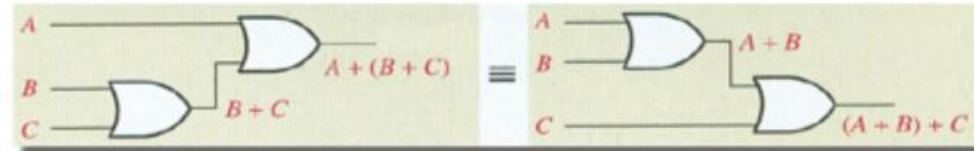


$$A \cdot B = B \cdot A$$

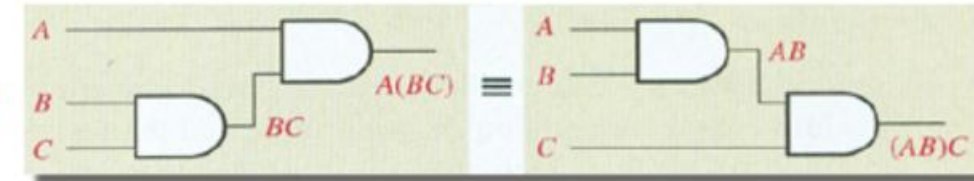


◆ Associative Laws

$$A + (B + C) = (A + B) + C$$



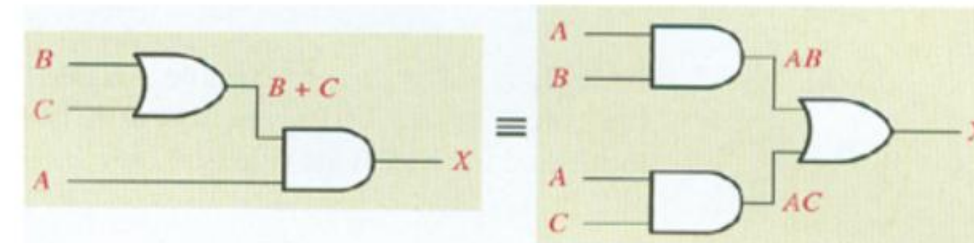
$$A \cdot (B \cdot C) = (A \cdot B) \cdot C$$



◆ Distributive Law

$$A \cdot (B + C) = A \cdot B + A \cdot C$$

$$A(B + C) = AB + AC$$



Rules

1. $A + 0 = A$

2. $A + 1 = 1$

3. $A \cdot 0 = 0$

4. $A \cdot 1 = A$

5. $A + A = A$

6. $A + \bar{A} = 1$

7. $A \cdot A = A$

8. $A \cdot \bar{A} = 0$

9. $\bar{\bar{A}} = A$

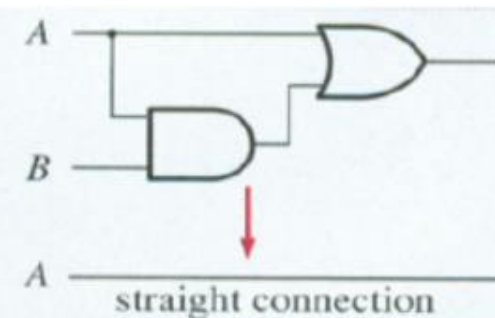
10. $A + AB = A$

11. $A + \bar{A}B = A + B$

12. $(A + B)(A + C) = A + BC$

- ◆ Rules 1 to 9 are obvious.
- ◆ Rule 10: $A + AB = A$

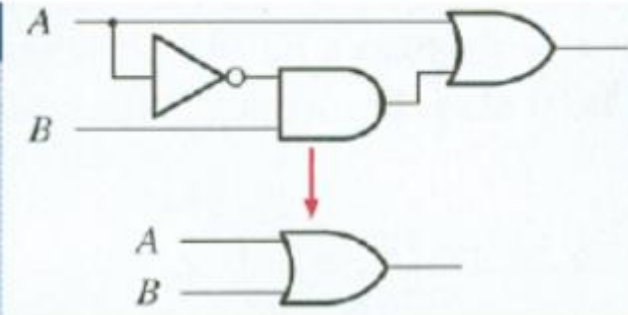
A	B	AB	$A + AB$
0	0	0	0
0	1	0	0
1	0	0	1
1	1	1	1



Rules

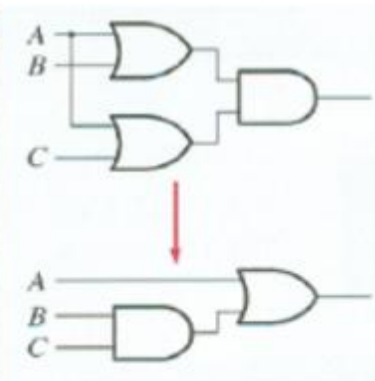
◆ Rule 11: $A + \overline{A}B = A + B$

A	B	$\overline{A}B$	$A + \overline{A}B$	$A + B$
0	0	0	0	0
0	1	1	1	1
1	0	0	1	1
1	1	0	1	1



◆ Rule 12: $(A + B)(A + C) = A + BC$

A	B	C	$A + B$	$A + C$	$(A + B)(A + C)$	BC	$A + BC$
0	0	0	0	0	0	0	0
0	0	1	0	1	0	0	0
0	1	0	1	0	0	0	0
0	1	1	1	1	1	1	1
1	0	0	1	1	1	0	1
1	0	1	1	1	1	0	1
1	1	0	1	1	1	0	1
1	1	1	1	1	1	1	1



Simplify Boolean Equation

- $(A + B)(A + C) = A + BC$
- This rule can be proved as follows:
- $(A + B)(A + C) = AA + AC + AB + BC$ (Distributive law)
 $= A + AC + AB + BC$ ($AA = A$)
 $= A(1 + C) + AB + BC$ ($1 + C = 1$)
 $= A \cdot 1 + AB + BC$
 $= A(1 + B) + BC$ ($1 + B = 1$)
 $= A \cdot 1 + BC$ ($A \cdot 1 = A$)
 $= A + BC$

DeMorgan's Theorem

- ◆ Theorem 1

$$\overline{(x + y)} = \bar{x} \cdot \bar{y}$$

Remember:

- ◆ Theorem 2

$$\overline{(x \cdot y)} = \bar{x} + \bar{y}$$

**“Break the bar,
change the operator”**

- ◆ DeMorgan's theorem is very useful in digital circuit design
- ◆ It allows **ANDs to be exchanged with ORs by using invertors**
- ◆ DeMorgan's Theorem can be extended to any number of variables.

$$\begin{aligned} F &= \overline{X \cdot Y} + \overline{P \cdot Q} && \leftarrow 2 \text{ NAND plus 1 OR} \\ &= \bar{X} + \bar{Y} + \bar{P} + \bar{Q} && \leftarrow 1 \text{ OR with some input invertors} \end{aligned}$$

$$z = \overline{(\bar{A} + C)} \cdot \overline{(B + \bar{D})} \quad \longrightarrow \quad z = A\bar{C} + \bar{B}D$$

That's all for today.....